SVX

SystemVerilog Extension Library

Class Catalog

Core Library and UVM Utilities

Version 2.1.3 beta

November 2026

Mark Glasser

Copyright 2026 Mark Glasser. Licensed under the Apache License, Version 2.0.

Contents

I Core Library **3**

1 Types **4**
1.1 svx_types — Standard Typedefs . . . 4
1.2 type_handle . . . 4
1.3 type_match . . . 6
1.4 typeid . . . 6

2 Traits **8**
2.1 Traits Classes . . . 8

3 Containers **11**
3.1 object — Base Object . . . 11
3.2 container — Container Base Classes . . . 11
3.3 vector . . . 12
3.4 map . . . 13
3.5 queue . . . 14
3.6 stack . . . 15
3.7 deque . . . 15
3.8 circ_buf — Circular Buffer . . . 16
3.9 pair / triple / quadruple . . . 16
3.10 sorter . . . 17
3.11 multimap . . . 18

4 Iterators **20**
4.1 Iterator Interfaces . . . 20
4.2 List Iterators . . . 21
4.3 Map Iterators . . . 23
4.4 Permutation Iterators . . . 24
4.5 range . . . 26
4.6 map_range . . . 27
4.7 multimap_iterator . . . 27

5 Algorithms **30**
5.1 algo — Core Algorithms . . . 30
5.2 algo2 — Dual-Iterator Algorithms . . . 31
5.3 accum — Accumulation Algorithms . . . 31
5.4 fcn — Function Objects . . . 32
5.5 predicate — Predicate Classes . . . 33

6 Lexer **35**

6.1 lexer_core — Lexical Analyzer . . . 35
6.2 ctypes — Character Classification . . . 36

7 Linked Structures **37**
7.1 node . . . 37
7.2 tree . . . 37
7.3 tree_iterator . . . 38

8 Set **40**
8.1 set . . . 40

9 Utilities **41**
9.1 msg — Message Interface . . . 41
9.2 msg macros — Convenience Message Macros . . . 41
9.3 ver — Version . . . 42

II UVM Utilities **43**

10 Command Line Processor **44**
10.1 Overview . . . 44
10.2 cl — Option Registration . . . 45
10.3 cl_parse — Parser . . . 46

11 Plugins **47**
11.1 plugins — UVM Plugins . . . 47

12 Agreements **48**
12.1 agreement — Shutdown Agreements . . . 48

13 Pipelined Testbenches **50**
13.1 Pipe Stages . . . 50

Part I

Core Library

Types

1.1 svx_types — Standard Typedefs

File: core/src/types/svx_types.svh

Standardized C-style integer typedefs, size and index types, process identifiers, and package-level Boolean constants. Provides a portable numeric vocabulary across the library.

Typedef	Base Type	Description
`int8_t`	`byte`	Signed 8-bit integer.
`uint8_t`	`byte unsigned`	Unsigned 8-bit integer.
`int16_t`	`shortint`	Signed 16-bit integer.
`uint16_t`	`shortint unsigned`	Unsigned 16-bit integer.
`int32_t`	`int`	Signed 32-bit integer.
`uint32_t`	`int unsigned`	Unsigned 32-bit integer.
`int64_t`	`longint`	Signed 64-bit integer.
`uint64_t`	`longint unsigned`	Unsigned 64-bit integer.
`int128_t`	`bit signed [127:0]`	Signed 128-bit integer (two-state).
`uint128_t`	`bit unsigned [127:0]`	Unsigned 128-bit integer (two-state).
`size_t`	`uint64_t`	Represents a container or array size.
`index_t`	`uint64_t`	Unsigned container index.
`signed_index_t`	`int64_t`	Signed container index, used for skip distances and offsets.
`pid_t`	`int unsigned`	Process identifier.

1.2 type_handle

File: core/src/types/type_handle.svh

Singleton type-handle mechanism. `type_handle#(T)` is a parameterized class that returns a unique singleton object for each distinct type `T`, enabling runtime type identity comparison. `type_container#(T)` wraps any value with its type handle for polymorphic storage.

`type_handle_base`

virtual class **extends** `object`

Non-parameterized base for all type handles.

Method	Return	Args	Description
`get_type_handle`	`type_handle_base`	—	Return the singleton type handle for this object (pure virtual).
`get_typename`	`string`	—	Return the name of the type (pure virtual).

`type_handle#(T)`

class **extends** `type_handle_base`

Parameterized singleton. One instance exists per distinct type T.

Method	Return	Args	Description
`get_type`	`type_handle#()`	—	Return the singleton type handle for T, creating it on the first call. Static.
`get`	`type_handle_base`	—	Return the singleton type handle for T as a base-class handle. Static.
`get_type_handle`	`type_handle_base`	—	Return this singleton cast to base type.
`get_typename`	`string`	—	Return the name of T, as given by `$typename`.

`type_container_base`

class

Non-parameterized base for typed containers; supports polymorphic storage and formatted printing of the contained value.

Method	Return	Args	Description
`val2str`	`string`	`string fmt=""`	Return the value formatted with fmt, or with the default format if fmt is empty (default: empty string).
`set_default_fmt`	void	`string fmt`	Set the default format used by val2str.
`get_typename`	`string`	—	Return the name of the contained type (default: empty string).
`get_type_handle`	`type_handle_base`	—	Return the type handle for the contained type (default: null).

`type_container#(T)`

class **extends** `type_container_base`

Holds a value of type T along with its type handle.

Constructor: `new(--)` — Initialise the type-handle singleton for T.

Method	Return	Args	Description
get_type_handle	type_handle_base	—	Return the type handle for T.
set	void	T x	Store a new value, replacing any existing one.
get	T	—	Retrieve the stored value.
val2str	string	string fmt=""	Format the value with $sformatf, using fmt or the default format.
get_typename	string	—	Return the name of T.

1.3 type_match

File: core/src/types/type_match.svh

Compile-time type-compatibility utilities. type_match#(T1,T2) provides static functions to check whether two type parameters are identical or whether one is derived from the other. The convenience macro `check_is_derived_from installs a const-initializer that will call $fatal if the constraint is violated at elaboration time.

type_match#(T1, T2)

class

Static type-comparison utilities for T1 and T2.

Method	Return	Args	Description
test_is_match	bit	—	True if T1 and T2 are the same type. Static.
is_match	bit	int line, string file	True if match; fatal error with location if not. Static.
test_is_derived_from	bit	—	True if T1 is derived from T2 (via $cast probe). Static.
is_derived_from	bit	int line, string file	True if derived; fatal error if not. Static.
fail_match	void	int line, string file	Issue $fatal reporting type mismatch. Static.
fail_derived	void	int line, string file	Issue $fatal reporting derivation failure. Static.

1.4 typeid

File: core/src/types/typeid.svh

Runtime type-classification for common SystemVerilog scalar types. typeid#(T) uses type_handle singletons to determine at run time whether T is integral, two-state, four-state, real, or string. Classification is limited to explicitly enumerated built-in types.

`typeid#(T)`

class

Static type-classification functions for built-in SV scalar types.

Method	Return	Args	Description
`is_int`	`bit`	—	True if T is any integral type. Static.
`is_int_fail`	`bit`	—	Same as is_int(); prints error and calls $finish if false. Static.
`is_two_state`	`bit`	—	True if T is integral and two-state (e.g., bit, int). Static.
`is_two_state_fail`	`bit`	—	Same but terminates on failure. Static.
`is_four_state`	`bit`	—	True if T is logic, reg, integer, or time. Static.
`is_four_state_fail`	`bit`	—	Same but terminates on failure. Static.
`is_real`	`bit`	—	True if T is real. Static.
`is_real_fail`	`bit`	—	Same but terminates on failure. Static.
`is_string`	`bit`	—	True if T is string. Static.
`is_string_fail`	`bit`	—	Same but terminates on failure. Static.

Traits

2.1 Traits Classes

File: core/src/containers/traits.svh

The traits system provides per-type constants and static functions used by all parameterized containers and algorithms. Every traits class defines an `empty_t` typedef, an `empty` constant, and static `equal()`, `compare()`, and `sort()` functions. The base class is `void_traits`; scalar integer families derive from `base_int_traits`.

`void_traits`

class **extends** `void_t`

Base traits class. empty is null (void_t). All type-flag localparams default to false.

Method	Return	Args	Description
`equal`	`bit`	`void_t a, b`	Always returns 1 (void objects are equivalent). Static.
`compare`	`int32_t`	`void_t a, b`	Returns 0 if equal (always), else 1. Static.
`sort`	void	`ref void_t vec[$]`	No-op sort. Static.
`check_trait`	`bit`	`bit trait, bit exp, int line, string file`	Assert trait==exp; call fail() if not. Static.
`fail`	void	`int line, string file`	Issue $fatal with optional file/line. Static.

`object_traits`

class **extends** `void_traits`

Traits for object-derived classes. Delegates equal()/compare() to object::compare().

Method	Return	Args	Description
`equal`	`bit`	`object a, b`	True if a.compare(b)==0. Static.
`compare`	`int32_t`	`object a, b`	Returns a.compare(b). Static.

Method	Return	Args	Description
`sort`	void	`ref object vec[$]`	No-op (objects are unordered). Static.

`class_traits#(T)`

class **extends** `void_traits`

Traits for arbitrary class handles. empty is null. Uses identity (==) for equality.

Method	Return	Args	Description
`equal`	`bit`	`T a, T b`	True if a==b (handle identity). Static.
`compare`	`int32_t`	`T a, T b`	Returns 0 if equal, else 1. Static.
`sort`	void	`ref T vec[$]`	No-op. Static.

`base_int_traits#(T)`

class **extends** `void_traits`

Parameterized base traits for integral scalar types. empty=T'(0). Provides arithmetic compare and built-in sort.

Method	Return	Args	Description
`equal`	`bit`	`T a, T b`	True if a==b. Static.
`compare`	`int32_t`	`T a, T b`	Returns 1 if a>b, -1 if a<b, 0 if equal. Static.
`sort`	void	`ref T vec[$]`	Sort using SystemVerilog built-in queue sort. Static.

`base_unsigned_int_traits#(T)`

class **extends** `base_int_traits#(T)`

Sets is_signed=0, is_unsigned=1. Convenience base for all unsigned integer traits typedefs.

`base_signed_int_traits#(T)`

class **extends** `base_int_traits#(T)`

Sets is_signed=1, is_unsigned=0. Convenience base for all signed integer traits typedefs.

`base_four_state_traits#(T)`

class **extends** `base_int_traits#(T)`

Traits base for four-state types (logic, reg, integer, time). Sets is_two_state=0, is_four_state=1.

integer_traits

class **extends** `base_four_state_traits#(integer)`

Traits for the SystemVerilog integer type (four-state, signed).

reg_traits / logic_traits / time_traits

class **extends** `base_four_state_traits#(reg)`

Traits for reg, logic, and time (all four-state, unsigned). logic_traits and time_traits are typedefs of reg_traits.

bit_vector_traits#(N)

class **extends** `base_int_traits#(bit[N-1:0])`

Traits for packed bit-vector types of width N. is_array=1; is_arithmetic=1.

real_base_traits#(T)

class **extends** `void_traits`

Traits for real/shortreal. Uses epsilon-based equality (2.2e-16). Sorts elements using an item comparator.

Method	Return	Args	Description
`equal`	`bit`	`T a, T b`	True if |a-b| <= epsilon. Static.
`compare`	`int32_t`	`T a, T b`	1 if a>b (and not equal), -1 if a<b, else 0. Static.
`sort`	void	`ref T vec[$]`	Sort with item comparator. Static.

string_traits

class **extends** `void_traits`

Traits for string. empty is an empty string. Uses lexicographic comparison and sorting.

Method	Return	Args	Description
`equal`	`bit`	`string a, b`	True if a==b. Static.
`compare`	`int32_t`	`string a, b`	Lexicographic: 1 if a>b, -1 if a<b, 0 if equal. Static.
`sort`	void	`ref string vec[$]`	Sort with item comparator. Static.

Containers

3.1 object — Base Object

File: core/src/containers/object.svh

Defines `void_t` (the empty base class, used as the "void" type throughout SVX) and `object` (adds virtual to_str, compare, and copy to void_t). All containers and tree nodes derive ultimately from `object`.

`void_t`

class

The empty class. Serves as the null base type and as the type of the "empty" element for void_traits.

`object`

class **extends** `void_t`

Base object providing virtual to_str(), compare(), and copy().

Method	Return	Args	Description
`to_str`	`string`	—	Return a human-readable string (default: "").
`compare`	`int`	`object obj`	Compare to obj; return 0 (default implementation).
`copy`	void	`object rhs`	Shallow-copy from rhs (default: no-op).

3.2 container — Container Base Classes

File: core/src/containers/container.svh

Abstract container base hierarchy. `container` defines size/clear/is_empty. `typed_container#(T,P)` adds a typed empty-element sentinel initialised from the traits class P at construction.

`container`

virtual class **extends** `void_t`

Abstract base for all containers. Defines the core size/clear/is_empty interface.

Method	Return	Args	Description
`size`	`size_t`	—	Return the number of elements (pure virtual).
`clear`	void	—	Remove all elements (pure virtual).
`is_empty`	`bit`	—	True if container holds no elements (pure virtual).

`typed_container#(T, P)`

virtual class **extends** `container`

Typed abstract base. Provides a typed empty-element sentinel initialised from the traits class P at construction.

Constructor: `new(--)` — Initialise the typed empty-element sentinel from the traits class P.

Method	Return	Args	Description
`size`	`size_t`	—	Overridable; returns 0 by default.
`is_empty`	`bit`	—	Overridable; returns 0 by default.
`clear`	void	—	Overridable; no-op by default.

3.3 vector

File: core/src/containers/vector.svh

Dynamic array backed by a SystemVerilog queue. Provides random read/write with automatic extension, shallow copy/clone, equality comparison, sort, and bulk append operations. The base class for queue, stack, and deque.

`vector#(T, P)`

class **extends** `typed_container#(T,P)`

Ordered, random-access dynamic array.

Method	Return	Args	Description
`create`	`this_t`	`vector_type list`	Static factory: create a vector preloaded from a queue literal.
`size`	`size_t`	—	Number of elements currently in the vector.
`is_empty`	`bit`	—	True when size()==0.
`clear`	void	—	Delete all elements; reset to empty state.
`extend`	void	`size_t sz`	Push sz empty elements, growing the vector by that amount.

Method	Return	Args	Description
`write`	void	`index_t idx, T t`	Write element at idx; auto-extends if idx >= size().
`read`	`T`	`index_t idx`	Read element at idx; returns the empty sentinel if out of range.
`copy`	void	`this_t vec`	Shallow copy from vec into this vector.
`clone`	`this_t`	—	Return a new vector that is a shallow copy of this one.
`compare`	`int`	`this_t v`	Returns 0 if equal to v, else non-zero.
`equal`	`bit`	`this_t v`	Element-wise equality using P::equal().
`sort`	void	—	Sort elements using P::sort().
`appendv`	void	`vector_type v`	Append a queue literal to this vector.
`appendc`	void	`T t`	Append a single element to this vector.
`append`	void	`this_t v`	Append all elements of vector v to this vector.

3.4 map

File: `core/src/containers/map.svh`

Class-based dynamic associative array. Allows sparse key-value storage allocated on demand and passed by reference. `singleton_map` provides a global instance accessible via `get_inst()`.

`map#(KEY, T, P)`

class **extends** `typed_container#(T,P)`

Associative array keyed by KEY, storing values of type T.

Method	Return	Args	Description
`get`	`T`	`KEY key`	Return item for key; returns the empty sentinel if key not found.
`insert`	`bit`	`KEY key, T item`	Insert (key,item). Returns 1 if key is new, 0 if duplicate (value overwritten).
`size`	`size_t`	—	Number of key-value pairs.
`is_empty`	`bit`	—	True when size()==0.
`delete`	`bit`	`KEY key`	Remove entry for key. Returns 1 on success, 0 if not found.
`clear`	void	—	Remove all entries.
`exists`	`bit`	`KEY key`	True if key exists in the map.
`copy`	void	`this_t rhs`	Shallow copy from rhs into this map.
`clone`	`this_t`	—	Return a new map that is a shallow copy of this one.

Method	Return	Args	Description
compare	int32_t	this_t m	Returns 0 if maps are equal, else non-zero.
equal	bit	this_t m	True if all keys and P::equal()-checked values match.
first	bit	ref KEY index	Move to first key (smallest); returns 0 if empty. For iterator use.
last	bit	ref KEY index	Move to last key (largest). For iterator use.
next	bit	ref KEY index	Advance to next key. For iterator use.
prev	bit	ref KEY index	Move to previous key. For iterator use.

singleton_map#(KEY, T, P)

class **extends** map#(KEY,T,P)

A map of which only one instance exists per type parameterization.

Method	Return	Args	Description
get_inst	this_t	—	Return the global singleton instance, creating it on the first call. Static.

3.5 queue

File: core/src/containers/queue.svh

FIFO queue derived from vector. put() inserts at the head; get() removes from the tail. fixed_size_queue adds a maximum-size constraint and a push-success status.

queue#(T, P)

class **extends** vector#(T,P)

FIFO queue: items enter at the front and leave from the back.

Method	Return	Args	Description
put	void	T t	Push item onto the head (front) of the queue.
get	T	—	Pop and return item from the tail; returns empty if queue is empty.
peek	T	—	Return the tail item without removing it.
is_empty	bit	—	True when size()==0.
clone	this_t	—	Return a shallow copy of this queue.

`fixed_size_queue#(T, P)`

class **extends** `queue#(T,P)`

Queue that rejects pushes when the maximum size is reached.

Constructor: `new(size_t n=1)` — Set maximum capacity to n (minimum 1).

Method	Return	Args	Description
`set_max_size`	void	`size_t n`	Update the capacity limit; clamps to current size if n is too small.
`get_max_size`	`size_t`	—	Return the current capacity limit.
`put`	void	`T t`	Push item; silently rejected if at capacity.
`last_push_succeeded`	`bit`	—	True if the most recent put() succeeded.
`is_full`	`bit`	—	True when size() >= max_size.
`copy`	void	`this_t q`	Copy contents and capacity from q.
`clone`	`this_t`	—	Return a shallow copy.
`equal`	`bit`	`vector#() v`	True if element contents and capacity match.

3.6 stack

File: core/src/containers/stack.svh

LIFO stack derived from vector. push() and pop() operate on the front (index 0).

`stack#(T, P)`

class **extends** `vector#(T,P)`

LIFO stack: top of stack is index 0.

Method	Return	Args	Description
`push`	void	`T t`	Push item onto the top of the stack.
`pop`	`T`	—	Remove and return the top item; returns empty if stack is empty.
`peek`	`T`	—	Return top item without removing it.
`is_empty`	`bit`	—	True when size()==0.
`clone`	`this_t`	—	Return a shallow copy of this stack.

3.7 deque

File: core/src/containers/deque.svh

Double-ended queue derived from vector. Supports push/pop at both ends, shuffle, and reverse.

`deque#(T, P)`

class **extends** `vector#(T,P)`

Double-ended queue with push/pop at both front and back.

Method	Return	Args	Description
`push_front`	void	`T t`	Insert item at the front.
`push_back`	void	`T t`	Insert item at the back.
`pop_front`	`T`	—	Remove and return front item.
`pop_back`	`T`	—	Remove and return back item.
`shuffle`	void	—	Randomly reorder elements.
`reverse`	void	—	Reverse the order of elements.
`clone`	`this_t`	—	Return a shallow copy of this deque.

3.8 circ_buf — Circular Buffer

File: core/src/containers/circ_buf.svh

Fixed-capacity circular (ring) buffer backed by a static array of size S, parameterised on element type T, traits P, and compile-time capacity S. Operates in one of two modes. In the default reject mode a push into a full buffer is refused with an error message. In overwrite mode a push into a full buffer silently overwrites the oldest item at the tail.

`circ_buf#(T, P, S)`

class **extends** `typed_container#(T,P)`

Fixed-size circular buffer holding S elements in FIFO ordering.

Constructor: `new(--)` — Calls clear() to initialise head, tail, and count.

Method	Return	Args	Description
`size`	`size_t`	—	Number of items currently in the buffer.
`is_empty`	`bit`	—	True when count==0.
`is_full`	`bit`	—	True when count==S.
`clear`	void	—	Reset head, tail, and count to zero.
`push`	void	`T t, circ_buf_mode_e mode`	Insert t; in REJECT_MODE prints error if full. In OVERWRITE_MODE the tail item is overwritten.
`pop`	`T`	—	Remove and return head item; prints error and returns empty if buffer is empty.

3.9 pair / triple / quadruple

File: core/src/containers/pair.svh

Lightweight tuple containers for binding two, three, or four values of potentially different types. All extend pair#(T1,T2) and derive from void_t (use void_traits when placing in a container).

`pair#(T1, T2)`

class **extends** `void_t`

Holds two values of types T1 and T2.

Constructor: `new(T1 a, T2 b)` — Initialise both fields.

Method	Return	Args	Description
`first`	`T1`	—	Return the first element.
`second`	`T2`	—	Return the second element.
`set_first`	void	`T1 t`	Replace the first element.
`set_second`	void	`T2 t`	Replace the second element.

`triple#(T1, T2, T3)`

class **extends** `pair#(T1,T2)`

Extends pair with a third element of type T3.

Constructor: `new(T1 a, T2 b, T3 c)` — Initialise all three fields.

Method	Return	Args	Description
`third`	`T3`	—	Return the third element.
`set_third`	void	`T3 t`	Replace the third element.

`quadruple#(T1, T2, T3, T4)`

class **extends** `triple#(T1,T2,T3)`

Extends triple with a fourth element of type T4.

Constructor: `new(T1 a, T2 b, T3 c, T4 d)` — Initialise all four fields.

Method	Return	Args	Description
`fourth`	`T4`	—	Return the fourth element.
`set_fourth`	void	`T4 t`	Replace the fourth element.

3.10 sorter

File: core/src/containers/sorter.svh

Generic sort implementation for SystemVerilog queues. The comparator is provided by the traits parameter P.

`sorter#(T, P)`

class

Sorts queues of type T using P::compare() as the element comparator.

Method	Return	Args	Description
`sort`	void	`ref T vec[$]`	Public entry point: sort vec in place. Static.
`qsort`	void	`ref T vec[$], int32_t l, int32_t r`	Sort the subrange [l, r] of vec recursively. Static.

3.11 multimap

File: core/src/containers/multimap.svh

A map in which each key may be associated with multiple values. Internally, each key maps to a `deque#(T,P)`, so insertion order is preserved within a key and values can be retrieved individually or as a group. `multimap` is built on top of `map#(KEY, deque_t, class_traits#(deque_t))`.

`multimap#(KEY, T, P)`

class **extends** `typed_container#(T,P)`

Container that maps each key to an ordered collection of values. Multiple values may be stored under the same key; they are kept in insertion order.

Constructor: `new(--)` — Construct an empty multimap.

Method	Return	Args	Description
`get`	`T`	`KEY key`	Return the last-inserted value for key, or P::empty if the key is absent.
`get_all`	`deque_t`	`KEY key`	Return a cloned deque of all values stored under key, in insertion order; returns the empty sentinel if the key is absent.
`get_map`	`map_t`	—	Return the internal map handle. Not intended for general use; required by multimap_iterator because SystemVerilog lacks friend access.
`insert`	void	`KEY key, T item`	Append item to the deque stored under key, creating the deque if needed.
`size`	`size_t`	—	Return the total number of items across all keys (O(k) in the number of distinct keys).
`size_nonzero`	`bit`	—	Return 1 if at least one item has been inserted; O(1) alternative to testing size() $>$ 0.

Method	Return	Args	Description
`contains`	`bit`	`KEY key`	Return 1 if key has at least one associated value, 0 otherwise.
`count`	`size_t`	`KEY key`	Return the number of values stored under key, or 0 if the key is absent.
`is_empty`	`bit`	—	True if no keys (and therefore no items) have been inserted.
`delete`	`bit`	`KEY key`	Remove all items stored under key; returns 1 on success, 0 if the key was not found.
`clear`	void	—	Remove all keys and values from the multimap.

Iterators

4.1 Iterator Interfaces

File: core/src/iterators/iterator_intf.svh

The iterator interface hierarchy. `iterator_intf_base` defines set/get/skip/size/is_empty. `fwd_intf` adds forward traversal (first/next/is_last/at_end). `bkwd_intf` adds backward traversal (last/prev/is_first/at_beginning). `random_intf` adds random selection.

`iterator_intf_base#(T, P)`

interface class

Base iterator interface. All iterators implement this.

Method	Return	Args	Description
`set`	void	`T t`	Write value T at the current position (pure virtual).
`get`	`T`	—	Read the value at the current position (pure virtual).
`skip`	`bit`	`signed_index_t d`	Move forward (d>0) or backward (d<0) by d steps (pure virtual).
`size`	`size_t`	—	Size of the bound container (pure virtual).
`is_empty`	`bit`	—	True if the iterator is unbound or the container is empty (pure virtual).

`fwd_intf#(T, P)`

interface class **extends** `iterator_intf_base#(T,P)`

Forward-traversal interface.

Method	Return	Args	Description
`first`	`bit`	—	Reset to first element; returns 0 if empty (pure virtual).
`next`	`bit`	—	Advance one step; may enter end state (pure virtual).
`is_last`	`bit`	—	True if pointing at the last element (pure virtual).

Method	Return	Args	Description
`at_end`	`bit`	—	True if past the last element (pure virtual).

`bkwd_intf#(T, P)`

interface class **extends** `iterator_intf_base#(T,P)`

Backward-traversal interface.

Method	Return	Args	Description
`last`	`bit`	—	Reset to last element; returns 0 if empty (pure virtual).
`prev`	`bit`	—	Move one step backward (pure virtual).
`is_first`	`bit`	—	True if pointing at the first element (pure virtual).
`at_beginning`	`bit`	—	True if before the first element (pure virtual).

`random_intf#(T, P)`

interface class **extends** `iterator_intf_base#(T,P)`

Random-access iterator interface.

Method	Return	Args	Description
`random`	`bit`	—	Select a random valid element (pure virtual).
`set_seed`	void	`int seed`	Set the RNG seed (pure virtual).
`set_default_seed`	void	—	Restore the default RNG seed (pure virtual).

4.2 List Iterators

File: `core/src/iterators/list_iterators.svh`

Iterators for `vector#()` and any class derived from it, including `stack`, `queue`, and `deque`. `list_iterator_base` provides the common binding and set/get/skip/size/is_empty logic. `list_iterator` traverses the container in either the forward or backward direction. `list_random_iterator` chooses a random item.

`list_iterator_base#(T, P)`

virtual class

Abstract base for list iterators. Manages the binding to a vector and provides the set/get/skip/size/is_empty operations shared by all list iterators.

Constructor: `new(list_t list=null)` — Optionally bind a vector.

Method	Return	Args	Description
`bind_list`	void	`list_t list`	Bind iterator to a vector (or unbind if null).
`set`	void	`T t`	Write T at the current index position.
`get`	`T`	—	Read value at the current index position.
`skip`	`bit`	`signed_index_t d`	Move index by d; clamps to valid range.
`size`	`size_t`	—	Return size of the bound vector.
`is_empty`	`bit`	—	True if unbound or vector is empty.

`list_iterator#(T, P)`

class **extends** `list_iterator_base#(T,P)` **implements** `fwd_intf#(T,P)`, `bkwd_intf#(T,P)`

Traverse a vector-derived container in either the forward or backward direction.

Constructor: `new(list_t list=null)` — Optionally bind a vector.

Method	Return	Args	Description
`first`	`bit`	—	Set index to 0; returns 0 if empty.
`next`	`bit`	—	Increment index; returns 0 if past end.
`is_last`	`bit`	—	True if index is at the last valid element.
`at_end`	`bit`	—	True if index >= size.
`last`	`bit`	—	Set index to last element; returns 0 if empty.
`prev`	`bit`	—	Decrement index; returns 0 if already at start.
`is_first`	`bit`	—	True if index == 0.
`at_beginning`	`bit`	—	True if index < 0 (before first element).

`list_random_iterator#(T, P)`

class **extends** `list_iterator_base#(T,P)` **implements** `random_intf#(T,P)`

Choose a random item in the bound vector.

Constructor: `new(list_t list=null)` — Optionally bind a vector; sets default RNG seed.

Method	Return	Args	Description
`random`	`bit`	—	Uniformly choose a random index in [0, size-1].

Method	Return	Args	Description
`set_seed`	void	`int seed`	Seed the RNG via $urandom(seed).
`set_default_seed`	void	—	Reset to the default seed (1).

4.3 Map Iterators

File: core/src/iterators/map_iterators.svh

Iterators for the map container. `map_iterator_base` provides a means for binding a map to an iterator and for setting and getting the item at the current position. `map_iterator` traverses the map in either the forward or backward direction. `map_random_iterator` provides random selection. Map keys are traversed in the natural sorted order of the underlying SV associative array.

`map_iterator_base#(KEY, T, P)`

virtual class

Abstract base for all map iterators. Provides a means for binding a map to the iterator and for setting and getting the item at the current iterator position.

Constructor: `new(map_t map=null)` — Optionally bind a map.

Method	Return	Args	Description
`bind_map`	void	`map_t m`	Bind iterator to map m.
`get_index`	`KEY`	—	Return the current key.
`size`	`size_t`	—	Return number of entries in the bound map.
`is_empty`	`bit`	—	True if unbound or map is empty.

`map_iterator#(KEY, T, P)`

class **extends** `map_iterator_base#(KEY,T,P)` **implements** `fwd_intf#(T,P)`, `bkwd_intf#(T,P)`

Traverse a map in either the forward or backward direction. In addition to the forward and backward traversal methods, provides `skip()`, which can move in either direction.

Constructor: `new(map_t map=null)` — Optionally bind a map.

Method	Return	Args	Description
`set`	void	`T t`	Update the value at the current key.
`get`	`T`	—	Return the value at the current key.
`get_index`	`KEY`	—	Return the current key.
`get_item`	`pair#(KEY, T)`	—	Return the current key and value as a pair.
`first`	`bit`	—	Move to the first (smallest) key; returns 0 if the map is empty.

Method	Return	Args	Description
`next`	`bit`	—	Advance to the next key; returns 0 if at the end of the map.
`is_last`	`bit`	—	True if current key equals the last key in the map.
`at_end`	`bit`	—	True if the iterator is past the last entry in the map.
`last`	`bit`	—	Move to the last (largest) key.
`prev`	`bit`	—	Move to the previous key; returns 0 if already at the first entry.
`is_first`	`bit`	—	True if current key equals first key in the map.
`at_beginning`	`bit`	—	True if the iterator is before the first entry in the map.
`skip`	`bit`	`signed_index_t d`	Skip d steps via repeated next()/prev() calls.

`map_random_iterator#(KEY, T, P)`

class **extends** `map_iterator_base#(KEY,T,P)` **implements** `random_intf#(T,P)`

Randomly selects a map entry.

Constructor: `new(map_t map=null)` — Optionally bind a map; set default RNG seed.

Method	Return	Args	Description
`set`	void	`T t`	Update value at current key.
`get`	`T`	—	Return value at current key.
`random`	`bit`	—	Select a uniformly random entry.
`set_seed`	void	`int seed`	Seed the RNG via $urandom(seed).
`set_default_seed`	void	—	Reset to default seed (1).

4.4 Permutation Iterators

File: core/src/iterators/permute_iterators.svh

Iterators that generate all permutations of a bound vector without modifying it. An internal index ordering tracks the current permutation; the bound vector itself is never changed. The original ordering of the bound vector is permutation 0. A vector of length n has $n!$ permutations. `get_nth(n)` returns the `n`th element of the current permutation; `get_permutation_vector()` returns the current element ordering. `permute_iterator` traverses all permutations sequentially in both directions; `permute_random_iterator` jumps to a randomly chosen permutation. Once a vector is bound, it should not be modified; call `initialize()` to reset if needed.

`permute_iterator_base#(T, P)`

class

Shared permutation state and algorithm. Maintains the current permutation ordering and the total permutation count, both computed and cached at bind time.

Constructor: `new(vec_t v=null)` — Optionally bind a vector and initialise the permutation state.

Method	Return	Args	Description
`bind_vector`	void	`vec_t v`	Bind a vector; calls initialize().
`size`	`size_t`	—	Size of the bound vector.
`is_empty`	`bit`	—	True if unbound or empty.
`get_nth`	`T`	`index_t n`	Return the nth element for the current permutation.
`get_permutation_index`	`longint`	—	Return the current permutation index (0-based).
`get_permutation_vector`	`vector#()`	—	Return the current permutation ordering as a vector of element indices.
`skip`	`bit`	`signed_index_t d`	Advance (positive d) or retreat (negative d) by d permutations.
`print`	void	—	Display the current permutation index and element ordering to stdout.

`permute_iterator#(T, P)`

class **extends** `permute_iterator_base#(T,P)` **implements** `fwd_intf#(T,P)`, `bkwd_intf#(T,P)`

Sequential bidirectional traversal of all permutations.

Constructor: `new(vec_t v=null)` — Optionally bind a vector.

Method	Return	Args	Description
`first`	`bit`	—	Reset to permutation 0.
`next`	`bit`	—	Advance to next permutation; returns 0 if already at last.
`is_last`	`bit`	—	True if at the last permutation.
`at_end`	`bit`	—	True if pix >= max_pix.
`last`	`bit`	—	Jump to the last permutation.
`prev`	`bit`	—	Return to the previous permutation.
`is_first`	`bit`	—	True if pix==0.
`at_beginning`	`bit`	—	True if pix<0.

`permute_random_iterator#(T, P)`

class **extends** `permute_iterator_base#(T,P)` **implements** `random_intf#(T,P)`

Jump to a randomly chosen permutation.

Constructor: `new(vec_t v=null)` — Optionally bind a vector; set default RNG seed.

Method	Return	Args	Description
`random`	`bit`	—	Select a uniformly random permutation index.
`set_seed`	void	`int seed`	Seed the RNG via $urandom(seed).
`set_default_seed`	void	—	Reset to default seed (1).

4.5 range

File: core/src/iterators/range.svh

A view over a contiguous sub-range [lb, ub] of an existing iterator. `range_base` stores and validates the bounds. `range#(T,P)` wraps a `fwd_intf` iterator and optionally a `bkwd_intf` if the underlying iterator supports it; backward access is silently unavailable otherwise.

range_base

virtual class

Abstract base that validates and stores index bounds; corrects inverted bounds automatically.

Constructor: `new(size_t size, index_t lb, index_t ub)` — Initialise the bounds, clamping to the container size and correcting any inversion.

Method	Return	Args	Description
`get_lower_bound`	`index_t`	—	Return lb.
`get_upper_bound`	`index_t`	—	Return ub.

range#(T, P)

class **extends** `range_base` **implements** `fwd_intf#(T,P)`, `bkwd_intf#(T,P)`

Sub-range iterator. Binds to a fwd_intf; optionally acquires backward capability via $cast.

Constructor: `new(iterator_intf_base#() it, index_t lb, ub)` — Bind to iterator it with the given range bounds; fails fatally if it does not support forward traversal.

Method	Return	Args	Description
`set`	void	`T t`	Set value at current iterator position.
`get`	`T`	—	Get value at current iterator position.
`size`	`size_t`	—	Number of elements in the range: ub-lb+1.
`is_empty`	`bit`	—	True if underlying iterator is null or empty.
`first`	`bit`	—	Move underlying iterator to position lb.
`next`	`bit`	—	Advance one step within range bounds.

Method	Return	Args	Description
is_last	bit	—	True if current position >= ub.
at_end	bit	—	True if position > ub.
last	bit	—	Move to position ub (requires bkwd_intf).
prev	bit	—	Move one step backward (requires bkwd_intf).
is_first	bit	—	True if current position == lb.
at_beginning	bit	—	True if position < lb.
skip	bit	signed_index_t d	Delegate to underlying fwd_iter.skip(d).

4.6 map_range

File: core/src/iterators/map_range.svh

Extends `range` for map iterators, adding `get_index()` to retrieve the current map key from within a sub-range view.

map_range#(KEY, T, P)

class **extends** `range#(T,P)`

Range view over a map iterator slice, exposing the current key.

Constructor: `new(map_iter_t it, index_t lb, ub)` — Delegate to range; retain reference to the map iterator.

Method	Return	Args	Description
get_index	KEY	—	Return the key at the current iterator position.

4.7 multimap_iterator

File: core/src/iterators/multimap_iterator.svh

Bidirectional iterator for `multimap`. Uses a dual-iterator design: a `map_iterator` advances across keys and a `list_iterator` traverses the values within each key's deque. The concrete `multimap_iterator` implements both `fwd_intf` and `bkwd_intf`.

multimap_iterator_base#(KEY, T, P)

virtual class

Abstract base that holds the map binding and the two internal iterators. Provides get_index(), size(), and is_empty().

Constructor: `new(multimap_t map=null)` — Optionally bind a multimap.

Method	Return	Args	Description
`bind_map`	void	`multimap_t m`	Bind the iterator to a multimap, initialising the internal map and list iterators.
`get_index`	`KEY`	—	Return the key associated with the current iterator position.
`size`	`size_t`	—	Return the total number of items in the bound multimap.
`is_empty`	`bit`	—	True if unbound or the multimap contains no items.

`multimap_iterator#(KEY, T, P)`

class **extends** `multimap_iterator_base#(KEY,T,P)` **implements** `fwd_intf#(T,P)`, `bkwd_intf#(T,P)`

Full bidirectional iterator over a multimap. Iterates all stored values in key order; within a key, values appear in insertion order.

Constructor: `new(multimap_t map=null)` — Optionally bind a multimap.

Method	Return	Args	Description
`get`	`T`	—	Return the value at the current position; returns P::empty if unbound or empty.
`set`	void	`T t`	Replace the value at the current position.
`get_index`	`KEY`	—	Return the key at the current position.
`get_item`	`pair#(KEY, T)`	—	Return the current key and value as a pair.
`first`	`bit`	—	Move to the first value in the map; returns 0 if empty.
`next`	`bit`	—	Advance one position; moves to the next value in the current deque, or to the first value of the next key. Returns 0 when past the end.
`is_last`	`bit`	—	True if the current position is the very last value in the multimap.
`at_end`	`bit`	—	True if the iterator is past the last value; always true when the multimap is empty.
`last`	`bit`	—	Move to the last value in the map; returns 0 if empty.
`prev`	`bit`	—	Move one position backward; moves to the previous value in the current deque, or to the last value of the preceding key. Returns 0 when before the beginning.

Method	Return	Args	Description
`is_first`	`bit`	—	True if the current position is the very first value in the multimap.
`at_beginning`	`bit`	—	True if the iterator is before the first value.
`skip`	`bit`	`signed_index_t d`	Move forward (d>0) or backward (d<0) by d positions; returns 0 if the limit is reached before all steps are taken.

Algorithms

5.1 algo — Core Algorithms

File: core/src/algorithms/algo.svh

Generic single-iterator algorithms. All functions are static and accept a `fwd_intf` iterator plus, where needed, a `predicate` or `fcn` object. find() positions the iterator at the first matching element rather than returning it.

`algo#(T, P)`

class

Collection of generic single-iterator algorithms.

Method	Return	Args	Description
`count`	`uint32_t`	`fwd_intf#() iter, predicate#(T) p`	Count elements for which p.is_true(). Static.
`all_of`	`bit`	`fwd_intf#() iter, predicate#(T) p`	True if predicate holds for every element. Static.
`none_of`	`bit`	`fwd_intf#() iter, predicate#(T) p`	True if predicate holds for no element. Static.
`any_of`	`bit`	`fwd_intf#() iter, predicate#(T) p`	True if predicate holds for at least one element. Static.
`min`	`T`	`fwd_intf#() iter`	Return minimum element using P::compare(). Static.
`max`	`T`	`fwd_intf#() iter`	Return maximum element using P::compare(). Static.
`find`	void	`fwd_intf#() iter, predicate#(T) p`	Position iter at first element where p.is_true(); no-op if not found. Static.

Method	Return	Args	Description
for_each	void	fwd_intf#() iter, fcn#(T) fn	Call fn.f(element) for every element. Static.

5.2 algo2 — Dual-Iterator Algorithms

File: core/src/algorithms/algo2.svh

Algorithms that operate on two iterators simultaneously. zip traverses both iterators in lock-step, calling fn.f(t1, t2) at each step until either iterator reaches its end.

algo2#(T1, P1, T2, P2)

class

Algorithms for paired traversal of two differently-typed iterators.

Method	Return	Args	Description
zip	void	fwd_ intf#(T1,P1) iter1, fwd_ intf#(T2,P2) iter2, fcn2#(T1,T2) fn	Traverse iter1 and iter2 in lock-step; call fn.f(t1,t2) at each step. Stops when either iterator ends. Static.

5.3 accum — Accumulation Algorithms

File: core/src/algorithms/accum.svh

Algorithms that carry a mutable accumulator A across iterations. accum operates on one iterator; accum2 operates on two in lock-step.

accum#(T, P, A)

class

Single-iterator accumulation algorithm.

Method	Return	Args	Description
accumulate	void	fwd_intf#() iter, accum_ fcn#(T,A) fn, ref A acc	Call fn.f(element, acc) for every element; acc is updated in place by fn. Static.

`accum2#(T1, P1, T2, P2, A)`

class

Dual-iterator accumulation algorithm.

Method	Return	Args	Description
`accumulate`	void	`fwd_intf#(T1,P1) iter1, fwd_intf#(T2,P2) iter2, accum_fcn2#() fn, ref A acc`	Traverse both iterators in lock-step; call fn.f(t1, t2, acc) at each step. Static.

5.4 fcn — Function Objects

File: core/src/algorithms/fcn.svh

Virtual function objects (SVX's substitute for C++ functors and lambdas). Users derive concrete classes and override `f()`. The hierarchy provides one-argument, two-argument, and accumulating variants.

`fcn_base`

virtual class

Empty base class for all function objects.

`fcn#(T)`

virtual class **extends** `fcn_base`

Abstract function object taking one argument of type T.

Method	Return	Args	Description
`f`	void	`T t`	Apply the function to t (pure virtual).

`fcn2#(T1, T2)`

virtual class **extends** `fcn_base`

Abstract function object taking two arguments of types T1 and T2.

Method	Return	Args	Description
`f`	void	`T1 t1, T2 t2`	Apply the function to (t1, t2) (pure virtual).

accum_fcn#(T, A)

virtual class **extends** `fcn_base`

Abstract function object taking one value and a mutable accumulator.

Method	Return	Args	Description
`f`	void	`T t, ref A a`	Apply the function; update accumulator a in place (pure virtual).

accum_fcn2#(T1, T2, A)

virtual class **extends** `fcn_base`

Abstract function object taking two values and a mutable accumulator.

Method	Return	Args	Description
`f`	void	`T1 t1, T2 t2, ref A a`	Apply the function to (t1, t2); update a in place (pure virtual).

5.5 predicate — Predicate Classes

File: core/src/algorithms/predicate.svh

Virtual predicate hierarchy. A predicate is a Boolean function over a single element T. Concrete predicate classes are composed with `and_pred`, `or_pred`, and `not_pred`.

predicate_base

virtual class

Empty base class for all predicate classes.

predicate#(T)

virtual class **extends** `predicate_base`

Abstract predicate: is_true() is pure virtual; is_false() delegates.

Method	Return	Args	Description
`is_true`	`bit`	`T t`	Return true if t satisfies the predicate (pure virtual).
`is_false`	`bit`	`T t`	Return !is_true(t).

and_pred#(T)

class **extends** `predicate#(T)`

True if both sub-predicates a and b are true.

Constructor: `new(predicate#(T) _a, _b)` — Store references to a and b.

Method	Return	Args	Description
`is_true`	`bit`	`T t`	Return a.is_true(t) && b.is_true(t).

`or_pred#(T)`

class **extends** `predicate#(T)`

True if at least one of the two sub-predicates is true.

Constructor: `new(predicate#(T) _a, _b)` — Store references to a and b.

Method	Return	Args	Description
`is_true`	`bit`	`T t`	Return a.is_true(t) || b.is_true(t).

`not_pred#(T)`

class **extends** `predicate#(T)`

True when the wrapped predicate is false.

Constructor: `new(predicate#(T) _p)` — Store reference to p.

Method	Return	Args	Description
`is_true`	`bit`	`T t`	Return p.is_false(t).

`always_true#(T)`

class **extends** `predicate#(T)`

Constant predicate that always evaluates to 1.

Method	Return	Args	Description
`is_true`	`bit`	`T t`	Always returns 1.

`always_false#(T)`

class **extends** `predicate#(T)`

Constant predicate that always evaluates to 0.

Method	Return	Args	Description
`is_true`	`bit`	`T t`	Always returns 0.

Lexer

6.1 lexer_core — Lexical Analyzer

File: core/src/lexer/lexer_core.svh

Breaks a string into tokens. To use the lexer, create a `lexer_core` object, call `start()` with the string to analyze, and then call `get_token()` repeatedly until it returns `TOKEN_EOL`. Each call returns a value of the enumerated type `token_t` (defined in `token_types.svh`). After each token, `get_lexeme()` returns its text, and for numeric tokens `get_token_descriptor()` returns details such as the size, base, and sign. Whitespace between tokens is skipped. The lexer does not handle comments.

```
lexer_core lex = new();
token_t    tok;

lex.start("size = 32h7fff_624c");
for(tok = lex.get_token(); tok != TOKEN_EOL; tok = lex.get_token())
  $display("%s: %s", tok.name(), lex.get_lexeme());
```

Numeric constants follow SystemVerilog conventions without the apostrophe, which command shells treat specially: an optional sign, an optional decimal size, a base letter (`b`, `o`, `d`, or `h`, either case), and digits, e.g. `32h7fff_624c` or `-100`. Underscores may appear between digits but not as the first character. Real constants use a decimal point and/or an exponent, and a number followed by a time unit (`s`, `ms`, `us`, `ns`, `ps`, `fs`) is a time constant. The words `on`, `true`, and `yes` and `off`, `false`, and `no` (all lower or all upper case) are returned as the tokens `TOKEN_ON` and `TOKEN_OFF`.

lexer_core

class

Constructor: `new()` — Create a lexer. Call `start()` before any other method.

Method	Return	Args	Description
`start`	void	`string s`	Initialize the lexer with the string to analyze.
`get_token`	`token_t`	—	Return the next token from the input; `TOKEN_EOL` at the end.
`get_lexeme`	`string`	—	Return the text of the most recent token. Empty for `TOKEN_EOL`.
`get_token_descriptor`	`token_descriptor`	—	Return details of the most recent numeric token.
`put_token`	void	`string lex`	Push a token back so it is returned again by the next `get_token()`. Pass the token's lexeme.

Method	Return	Args	Description
`lex_filename`	`token_t`	—	Take the rest of the input as a single raw value, such as a file path, without breaking it into tokens. Returns `TOKEN_STRING`, `TOKEN_ERROR` for an unterminated quote, or `TOKEN_EOL`.

`token_descriptor`

class

Details of the most recent numeric token, returned by `get_token_descriptor()`.

Field	Type	Description
`sign`	`byte`	Leading sign character, `+` or `-`; 0 if none.
`size`	`int unsigned`	Declared size of a sized constant; 0 if unsized.
`base`	`byte`	Base character: `d`, `h`, `o`, or `b` (either case).
`digits`	`string`	The digits of the value, without sign, size, or base.
`is_logic`	`bit`	Set if the digits include `x` or `z`.
`multiplier`	`real`	Scale factor for a time constant's unit, e.g. 1.0e-9 for `ns`.

6.2 ctypes — Character Classification

File: core/src/lexer/ctypes_macros.svh

Macros, modeled on the C `ctype` functions, that test a character for a characteristic. The lexer uses them, and they are available for any character processing.

Method	Return	Args	Description
`` `isalpha(c) ``	`bit`	—	Upper- or lower-case letter.
`` `isupper(c) ``	`bit`	—	Upper-case letter.
`` `islower(c) ``	`bit`	—	Lower-case letter.
`` `isdigit(c) ``	`bit`	—	Decimal digit 0–9.
`` `isxdigit(c) ``	`bit`	—	Hexadecimal digit 0–9, a–f, A–F.
`` `isodigit(c) ``	`bit`	—	Octal digit 0–7.
`` `isalnum(c) ``	`bit`	—	Letter or decimal digit.
`` `isspace(c) ``	`bit`	—	Whitespace character.
`` `isblank(c) ``	`bit`	—	Blank (space) character.
`` `ispunct(c) ``	`bit`	—	Punctuation character.
`` `isprint(c) ``	`bit`	—	Printable character: punctuation, letter, digit, or blank.
`` `islogic(c) ``	`bit`	—	Four-state logic digit: `x`, `X`, `z`, or `Z`.

Linked Structures

7.1 node

File: core/src/linked/node.svh

A named, markable container node extending `object`. The name field supports hierarchical naming in the tree. The mark/unmark mechanism supports traversal algorithms that need to track visited nodes.

`node`

class **extends** `object`

Named, markable base node for tree structures.

Constructor: `new(string nm="")` — Initialise name to nm.

Method	Return	Args	Description
`set_name`	void	`string nm`	Set the node name.
`get_name`	`string`	—	Return the node name.
`mark`	void	—	Set the mark flag to 1.
`unmark`	void	—	Clear the mark flag to 0.
`get_mark`	`bit`	—	Return the current mark value.
`is_marked`	—	—	Return the mark flag (equivalent to get_mark()).

7.2 tree

File: core/src/linked/tree.svh

Classic tree data structure. Each node has exactly one parent (except the root, which has none), a name, and a map of named children. Each node is a typed container that can hold an object of arbitrary type. Hierarchical path names use the dot (.) as the separator; the `find()` method locates any named node by dot-separated path.

`tree`

class **extends** `node`

N-ary tree node with named children, full-path support, and hierarchical search.

Constructor: `new(string nm, tree p)` — Initialise the node with name nm, link to parent p, and register as a child of p.

Method	Return	Args	Description
`insert`	void	`tree t`	Add t as a named child of this node; error if name duplicate.
`get_parent`	`tree`	—	Return the parent node handle.
`get_full_name`	`string`	—	Return the fully-qualified dotted path name.
`first_child`	`bit`	`ref string nm`	Set nm to the name of the first child; returns 0 if none.
`next_child`	`bit`	`ref string nm`	Advance nm to the next sibling name; returns 0 if none.
`num_children`	`size_t`	—	Return the number of direct children.
`size`	`size_t`	—	Return the total number of nodes in this subtree (including this node), counted recursively.
`get_children`	`deque#()`	—	Return a deque of all direct child handles.
`get_child`	`tree`	`string nm`	Return the named child; null if not found.
`find`	`tree`	`string path`	Locate a node by dot-separated path. Returns null on parse error or not found.
`mark_all`	void	—	Recursively mark all nodes in this subtree.
`unmark_all`	void	—	Recursively unmark all nodes in this subtree.

7.3 tree_iterator

File: core/src/linked/tree_iterator.svh

Pre-order depth-first iterator over a tree. `size()` counts all reachable nodes from the bound root.

`tree_iterator#(T, P)`

class **implements** `fwd_intf#(T,P)`

Forward-only pre-order DFS iterator over a tree.

Constructor: `new(tree t=null)` — Bind to tree t.

Method	Return	Args	Description
`size`	`size_t`	—	Return the total count of nodes reachable from the bound root. Note: this call alters node marks as a side effect.
`is_empty`	`bit`	—	True if no tree is bound.
`set`	void	`tree t`	Has no effect; tree traversal does not support element replacement.

Method	Return	Args	Description
`get`	`tree`	—	Return the tree node at the top of the stack.
`first`	`bit`	—	Begin traversal at the root.
`next`	`bit`	—	Advance to the next node in pre-order.
`is_last`	`bit`	—	True when exactly one node remains on the stack.
`at_end`	`bit`	—	True when the stack is empty.
`skip`	`bit`	`signed_index_t d`	Skip d nodes forward (negative d not supported).

Set

8.1 set

File: core/src/set/set_impl.svh

Mathematical set. All set operations return a new set and do not mutate their operands.

set#(T, P)

class **extends** `typed_container#(T,P)`

Mathematical set with insert, membership test, and set-algebra operations.

Constructor: `new(--)` — Allocate the internal map.

Method	Return	Args	Description
`size`	`size_t`	—	Return the number of members.
`is_empty`	`bit`	—	True when the set has no members.
`clear`	void	—	Remove all members.
`insert`	void	`T item`	Add item to the set (duplicates ignored via map semantics).
`contains`	`bit`	`T item`	True if item is a member.
`intersection`	`set_t`	`set_t s`	Return a new set of elements in both this and s.
`set_union`	`set_t`	`set_t s`	Return a new set of all elements in this or s (or both).
`difference`	`set_t`	`set_t s`	Return a new set of elements in this but not in s.
`to_vector`	`vector#()`	—	Return a vector containing all set members in key order.
`from_vector`	void	`vector#() vec`	Insert all elements of vec into the set (skipping duplicates).

Utilities

9.1 msg — Message Interface

File: core/src/utils/msg.svh

A simple, uniform interface for printing diagnostic messages. All methods are static, so no instance is required. Each method prints a formatted message prefixed with its severity tag and the caller-supplied identifier.

msg

class

Static message-printing class. Wraps `$error`, `$warning`, `$info`, and `$fatal`.

Method	Return	Args	Description
`error`	void	`string id, string txt`	Print an error message via $error. Static.
`warning`	void	`string id, string txt`	Print a warning message via $warning. Static.
`info`	void	`string id, string txt`	Print an informational message via $info. Static.
`fatal`	void	`string id, string txt`	Print a fatal message and abort simulation via $fatal(1,...). Static.

9.2 msg macros — Convenience Message Macros

File: core/src/utils/msg_macros.svh

Preprocessor convenience macros that forward directly to the `msg` class static functions. Using the macros avoids writing `msg::` at every call site and keeps diagnostic output consistent across the library.

msg macros

macros

Four single-statement macros, one per severity level. Each expands to the corresponding `msg` static-function call.

Method	Return	Args	Description
`` `info_msg(id, txt) ``	void	—	Expands to `msg::info(id,txt)`.
`` `warning_msg(id, txt) ``	void	—	Expands to `msg::warning(id,txt)`.
`` `error_msg(id, txt) ``	void	—	Expands to `msg::error(id,txt)`.
`` `fatal_msg(id, txt) ``	void	—	Expands to `msg::fatal(id,txt)`.

9.3 ver — Version

File: `core/src/utils/version.svh`

Exposes the library version number through static accessor functions. Version components are defined by macros in the library header and retrieved at run time through this class.

`ver`

class

Static version-query class for the SVX library.

Method	Return	Args	Description
`get_major_version`	`int unsigned`	—	Return the major version number. Static.
`get_minor_version`	`int unsigned`	—	Return the minor version number. Static.
`get_fix_version`	`int unsigned`	—	Return the fix/patch version number. Static.
`get_release_mode`	`string`	—	Return the release mode string (e.g., `beta`). Static.
`get_version_string`	`string`	—	Return the full version string. Static.
`print_banner`	void	—	Display a formatted version banner to stdout. Static.

Part II

UVM Utilities

Command Line Processor

10.1 Overview

File: uvm/src/README-cl.md

The command line processor reads the plusargs on the simulator command line — arguments beginning with + — checks each one against a table of options, converts its value to the option's type, and puts it into the UVM resource database where testbench code can retrieve it. Because every option is registered in one table, the table is a complete list of the options a testbench accepts.

Use Model

Register each option with `cl#(T)::add()` or `cl#(T)::add_with_default()`, where T is the type of the option's value, then call `parse()` on the singleton parser. Do both before calling `run_test()` so every phase, including `build_phase`, has access to the values. Options can be registered anywhere — in any module, package, or class — and in any order, as long as it is before the command line is parsed.

```
module top;

  function automatic void preliminaries();
    cl_parse clp = cl_parse::get();
    cl#(uint32_t)::add_with_default("size", 4, "Size of things");
    cl#(bit)::add("x_mode", "Enable X mode");
    cl#(uint32_t)::add("base", "Base address", "%8x");
    clp.parse();
  endfunction

  initial begin
    preliminaries();
    run_test();
  end

endmodule
```

The parser puts each option into the resource database under the scope `CL::`, which keeps command line options from conflicting with other resources. Retrieve a value with the standard `uvm_resource_db` interface, using the same type the option was registered with:

```
uint32_t size;
uvm_resource_db#(uint32_t)::read_by_name("CL::", "size", size, this);
```

Command Line Syntax

An option takes one of two forms, `+name` or `+name=value`, with no spaces around the `=`. The form `+name` is a switch and is valid only for `bit` options: it turns the option on. A `bit` option can also be set explicitly:

Value	Meaning
`1, on, true, yes`	Turn the switch on. Equivalent to `+name`. Keywords may be all lower or all upper case.
`0, off, false, no`	Turn the switch off.

A value must fit the option's type; `+bias=2000` is rejected for a `byte` option even though it is a valid number. Integers may be negative, and may be sized and given in any base — `h`, `o`, `b`, or `d`, in either case — as in SystemVerilog but without the apostrophe, which command shells use for their own purposes. Underscores may separate digits:

```
+base=32hf77_7003c
+mask=12b011_000_111_010
+mode=12o6263
+bias=-100
```

`string` values are taken exactly as written, so file paths and other text need no quoting. If an option appears more than once, the last value wins. Plusargs that are not registered options — simulator and UVM options, or plusargs used by other code — are ignored. If any registered option has an invalid value, the parser reports an error and puts none of the command line values into the resource database, and UVM stops the run at the end of elaboration.

Supported types are `string`, `real`, `bit`, `byte`, `shortint`, `int`, `longint`, and their `unsigned` forms, along with the aliases `int8_t` through `int64_t` and `uint8_t` through `uint64_t`.

Built-in Options

The parser registers three options of its own. Their names are reserved.

Option	Effect
`+svx_help`	Print the SVX version, the option table with descriptions and defaults, and a summary of the command line syntax, then end the simulation. Works even when other options on the command line are invalid.
`+svx_cl_dump`	Print the option table after parsing, then continue.
`+plugin=<type>`	Install a UVM plugin; may be given more than once. See the Plugins chapter.

10.2 cl — Option Registration

File: uvm/src/cl.svh

Static functions for adding options to the option table. The type parameter is the type of the option's value. The option name is given without the leading `+`. An option registered after the command line has been parsed is ignored with a warning, and registering the same name twice is an error.

`cl#(T)`

class

Method	Return	Args	Description
`add`	void	`string name, string desc="", string fmt=""`	Add option `name` of type T to the table. `desc` is a one-line description shown by `+svx_help` and `+svx_cl_dump`. `fmt` optionally sets the format used to print the value. Static.
`add_with_default`	void	`string name, T def, string desc="", string fmt=""`	Add an option with a default value. The default goes into the resource database immediately and is replaced by a value given on the command line. Static.

10.3 cl_parse — Parser

File: `uvm/src/cl_parse.svh`

The command line parser is a singleton. Obtain it with `get()` and call `parse()` once, after all options are registered.

`cl_parse`

class

Method	Return	Args	Description
`get`	`cl_parse`	—	Return the singleton parser, creating it and registering the built-in options on first call. Static.
`parse`	void	—	Parse the command line, validate each registered option, and put the values into the resource database. Parsing happens only once; later calls issue a warning and do nothing.

Plugins

11.1 plugins — UVM Plugins

File: uvm/src/plugins.svh

A plugin is a UVM component that is instantiated under `uvm_root` instead of inside the testbench hierarchy. It can be any kind of component and perform any function: monitoring, coverage collection, debugging, reporting. Plugins are selected on the command line with `+plugin=<type>`, so they can be added to a run without changing the testbench.

A plugin class must be registered with the UVM factory (`uvm_component_utils`) and must be creatable without a parent of its own; it may create subordinate components. Each plugin is created with the factory, using the type name as the instance name, and then goes through every phase like any other component.

Install plugins after parsing the command line and before calling `run_test()`:

```
initial begin
  cl_parse clp = cl_parse::get();
  clp.parse();          // parse the command line
  plugins::install();   // instantiate +plugin=<type> components
  run_test();
end
```

```
% <simulator> ... +plugin=my_plugin +plugin=my_other_plugin
```

plugins

class

Static functions for installing plugins.

Method	Return	Args	Description
`install`	void	—	Instantiate a component for each `+plugin=<type>` on the command line. Reports an error for a type the factory cannot create. Must run after `parse()` and before `run_test()`. Static.

Agreements

12.1 agreement — Shutdown Agreements

File: uvm/src/agreement.svh, uvm/src/README-agreement.md

An agreement is a lightweight synchronization primitive for threads in different objects. Participating objects vote to agree or disagree; a thread that calls `wait_for_agreement()` blocks until no participant disagrees. Participants may change their votes at any time. The agreement is described in Glasser, "Shutdown with Agreements in a UVM Testbench," SNUG 2017.

A common use is deciding when to end a test. A top-level component creates an agreement, puts it into the resource database, and waits on it in `run_phase` while holding an objection. Stimulus generators, scoreboards, and other components retrieve the agreement and disagree while they have work to do and agree when they are finished. When every participant agrees, the waiting component drops its objection.

```
// In the environment
ok_to_shutdown = new("ok_to_shutdown");
uvm_resource_db#(agreement)::set({get_full_name(), ".*"},
                                 "ok_to_shutdown", ok_to_shutdown, this);
...
phase.raise_objection(this);
ok_to_shutdown.wait_for_agreement(this);
phase.drop_objection(this);

// In a participating component
ok_to_shutdown.disagree(this);
... do the work ...
ok_to_shutdown.agree(this);
```

Participants should disagree at the start of `run_phase`, before anything that blocks, or earlier in a function phase. `wait_for_agreement()` yields once before testing, so that participants starting in the same time step can register their disagreement first. Any number of threads may wait on the same agreement.

agreement

class **extends** `uvm_object`

Constructor: `new(string name = "agreement")` — Create an agreement with no votes.

Method	Return	Args	Description
`agree`	void	`uvm_object obj, int lineno=0`	Record that obj agrees. Agreeing has an effect only if obj previously disagreed.

Method	Return	Args	Description
`disagree`	void	`uvm_object obj, int lineno=0`	Record that obj disagrees.
`wait_for_agreement`	void	`uvm_object obj, int lineno=0`	Block until no participant disagrees. Task.
`clear`	void	`uvm_object obj, int lineno=0`	Discard all votes.
`num_disagreements`	`uint32_t`	—	Return the number of participants that currently disagree.
`set_debug / clr_debug`	void	—	Turn debug messages on or off for this agreement.
`set_debug_all / clr_debug_all`	void	—	Turn debug messages on or off for every agreement. Static.

In each method, `obj` identifies the participant, normally `this`. The optional `lineno`, typically `` `__LINE__ ``, is included in debug messages.

Pipelined Testbenches

13.1 Pipe Stages

File: uvm/src/pipe_stage_base.svh, uvm/src/pipe_stage.svh, uvm/src/README-pipe.md

Base classes for building a testbench as a chain of stages that run concurrently. Each stage looks like a driver to its upstream neighbor and like a sequencer to its downstream neighbor, so stages connect with ordinary sequence-item ports. A stage begins work when an item arrives from upstream and at some point sends an item downstream — the same item or a different one. Items in and out need not correspond one to one.

`pipe_stage_base` is used directly for the first stage, which creates items itself; `pipe_stage` is used for every later stage. Connect each stage's `seq_item_port` to the `seq_item_export` of the stage before it. A stage sends an item downstream by calling `sqr.execute_item()` from `process_item()`.

Stages decide when the pipeline is finished with an agreement that must be in the resource database under the name `pipeline_done`, visible to every stage. Each `pipe_stage` disagrees when an item arrives and agrees when it becomes idle; a first stage derived from `pipe_stage_base` must vote itself: disagree in `build_phase` and agree after sending its last item. The environment waits on the agreement in `run_phase`.

`pipe_stage_base#(SEQ)`

class **extends** `uvm_component`

Base for all stages, and the class for the first stage. SEQ is the sequence item type. Derived classes use the protected members `sqr` (the stage's sequencer, for sending items downstream) and `pipeline_done` (the agreement).

Constructor: `new(string name, uvm_component parent)` — Standard component constructor.

Method	Return	Args	Description
`process_item`	void	`SEQ item`	Hook for handling an item, commonly by sending it downstream with `sqr.execute_item()`. Virtual task.

Member	Type	Description
`seq_item_export`	`uvm_seq_item_pull_export#(SEQ)`	Connect the downstream stage's `seq_item_port` here.

`pipe_stage#(SEQ)`

class **extends** `pipe_stage_base#(SEQ)`

Base for every stage after the first. `run_phase` runs two threads: one pulls items from upstream into an internal FIFO, the other takes items from the FIFO and calls `process_item()`. Pulling from upstream completes as soon as the item is in the FIFO, so a bounded FIFO is what lets a slow stage hold back the stages before it.

Constructor: `new(string name, uvm_component parent, int32_t size = 0)` — size is the depth of the internal FIFO; 0 means unbounded.

Method	Return	Args	Description
`process_item`	void	`SEQ item`	Inherited hook; called for each item taken from the FIFO. Virtual task.
`is_idle`	`bit`	—	Return 1 when the stage has no work left; the stage then agrees to shut down. The default is true when the FIFO is empty. Override it for stages that hold work across calls, such as a stage that collects several items before sending one. Virtual.

Member	Type	Description
`seq_item_port`	`uvm_seq_item_pull_port#(SEQ)`	Connect to the upstream stage's `seq_item_export`.